

DevOps Pro — Kubernetes

Архитектура · Workloads · Networking · Storage · Security

Author: Aliaksandr Tsimokhau

Agenda

Foundations

Background · Architecture · kubectl

Workloads

Pods · Namespaces · Deployments

Networking

Services · Ingress · Discovery

Data & State

Storage · Config · Stateful

Security & API

RBAC · API · Security

Scheduling, Scaling & Batch

Scheduling · Autoscaling · Batch

Part 1: Kubernetes Background

Что такое оркестратор и почему именно Kubernetes

Kubernetes — оркестратор контейнеризованных приложений

- **Orchestrator** — система, которая разворачивает приложения и реагирует на изменения
- Kubernetes умеет:
 - разворачивать приложения
 - масштабировать вверх и вниз по нагрузке
 - **self-heal** — перезапускать упавшие части
 - выполнять zero-downtime rolling updates и rollbacks
- Происхождение: создан в Google, в 2014 передан в CNCF
- Написан на Go, лицензия Apache 2.0, версия 1.0 в июле 2015

Три поколения платформ выполнения

VM

- Тяжёлая
- Запуск — минуты
- Сильная изоляция
- Старая модель

Container

- Легче VM
- Запуск — секунды
- Слабее изоляция
- Платформо-зависим

Wasm

- Очень лёгкий
- Запуск — миллисекунды
- Deny-by-default
- Build once, run anywhere

Kubernetes умеет запускать **все три** (Wasm — через `RuntimeClass` + containerd shim)

Cloud-native ≠ "просто запустил в облаке"

Cloud-native приложения обладают свойствами:

- **auto-scaling** — автоматически масштабируются
- **self-healing** — восстанавливаются после сбоев
- **automated updates and rollbacks** — могут обновляться без простоя
- **observability** — экспортируют метрики и логи

Просто запустить классическое монолитное приложение в AWS — это **не cloud-native**.

Как проектировать приложение под Kubernetes

- **Stateless first** — состояние выносится во внешние БД / очереди / object storage
- **Config outside image** — конфиг и секреты приходят через ConfigMap / Secret
- **Scale-out friendly** — приложение должно переживать несколько одинаковых реплик
- **Disposable** — корректно стартует, останавливается и переживает пересоздание Pod
- **Observable** — пишет логи в stdout/stderr и экспортирует health/metrics

Хороший mental model: Kubernetes любит процессы, которые легко клонировать, заменить и наблюдать.

Kubernetes и Docker — что случилось

- Ранние версии Kubernetes использовали **Docker** как runtime
- Docker раздулся → появился **CRI (Container Runtime Interface)**
- Kubernetes 1.24 (2022) удалил поддержку Docker как runtime
- Сейчас стандарт — **containerd** (минималистичный, отделён от Docker Engine)
- RedHat OpenShift использует **CRI-O**
- Все три (Docker, containerd, CRI-O) поддерживают **OCI**-стандарт — образы взаимозаменяемы

Time to make hands dirty — Lab 01

Install a local Kubernetes cluster

Поднимем рабочий кластер на ноутбуке (kind / Docker Desktop / minikube).
Без кластера дальше не получится.

 **labs/lab-01-install-cluster.md**

Part 2: Principles of Operation

Архитектура: control plane, worker nodes, declarative model

Cluster = control plane + data plane

- **Cluster** — один или более узлов (физические машины, VM или cloud-инстансы)
- Два типа узлов:
 - **Control plane** — "мозг" кластера
 - **Data plane (worker nodes)** — здесь крутятся ваши приложения
- Control plane **обязательно Linux**; data plane nodes — Linux или Windows
- Для HA — **3 или 5** control plane nodes
- Большие кластеры могут вынести etcd на отдельные узлы

Control plane components

kube-apiserver

- Фронт-энд кластера, RESTful HTTPS
- Все запросы — через него
- AuthN + AuthZ + Admission

etcd (cluster store)

- Распределённая KV-БД на RAFT
- Хранит desired state
- Источник истины для кластера

kube-scheduler

- Watches API server
- Assigns pods to nodes
- Filter + score алгоритм

controller-manager

- Запускает controllers
- Deployment, ReplicaSet, StatefulSet и др.
- Reconciliation loops

cloud-controller-manager

- Интеграция с облаком
- LoadBalancers, volumes, nodes
- AWS / GCP / Azure / Civo

etcd — память кластера

- Распределённая KV-БД, основа Kubernetes
- Если API server — мозг, то etcd — память
- **Нечётное число реплик** (3, 5, 7) — для избежания split-brain
- При split-brain etcd уходит в **read-only**: приложения работают, но изменения в кластер не применяются
- Алгоритм консенсуса — **RAFT**

⚠ Бэкап etcd — критически важен. Потерять etcd = потерять весь кластер.

Data plane (worker nodes) — где живут приложения

- **kubelet** — главный агент K8s на узле
 - Watches API server для новых задач
 - Командует runtime'у запускать контейнеры
 - Репортит статус назад в API server
- **Container runtime** (containerd / CRI-O) — низкоуровневая работа с контейнерами
- **kube-proxy** — сетевой "мозг" узла
 - Раздаёт IP подам
 - Лёгкая load-balancing логика

Declarative model — главная идея Kubernetes

1. **Observed state** — что есть сейчас
2. **Desired state** — что вы хотите
3. **Reconciliation** — процесс приведения observed к desired

Как работает:

1. Описываете желаемое состояние в YAML
2. POST в API server
3. etcd сохраняет как "запись о намерении"
4. Controller замечает расхождение
5. Применяет изменения
6. Reconciliation loop **никогда не останавливается**

Packaging apps for Kubernetes

Каждое приложение должно быть упаковано:

1. **Container** обрачивает приложение и его зависимости
2. **Pod** обрачивает контейнер → может работать в K8s
3. **Deployment / StatefulSet / DaemonSet** обрачивает Pod → добавляет self-healing, scaling, rollouts

⚠ Pod — это минимальная единица **развёртывания** в Kubernetes, **НЕ** контейнер!

Pods и Services — preview

- **Pods are mortal** — они умирают, и K8s заменяет их **новыми** с новыми IP
- Это создаёт IP churn → клиенты не могут полагаться на IP подов
- **Service** = стабильная точка входа для группы подов
 - Стабильное DNS-имя
 - Стабильный IP
 - Лёгкий round-robin load balancing
 - Только к healthy pods
- Service знает, к каким подам слать трафик, через **labels**

Part 3: Getting Kubernetes & kubectl

Установка кластеров и работа с kubeconfig

Варианты получения кластера

Тип	Примеры	Когда использовать
Local	Docker Desktop, kind, minikube, k3d	Разработка, обучение
Managed	GKE, EKS, AKS, DOKS	Production в облаке
Self-hosted	kubeadm	On-prem, кастомные требования
PaaS	OpenShift, Rancher, k3s	Enterprise / edge

kubectl — CLI Kubernetes

- Конвертирует user-friendly команды в HTTPS REST запросы
- Версия kubectl должна быть в пределах **±1 minor** от версии кластера
- Читает **kubeconfig** для связи с кластером и credentials

```
kubectl version --client  
kubectl cluster-info  
kubectl get nodes  
kubectl api-resources
```

kubeconfig — где живут credentials

Файл `~/.kube/config` содержит три блока:

clusters

- name
- server (URL API)
- certificate-authority-data

contexts

- name
- cluster + user + namespace
- `current-context` выбирает default

users

- name
- credentials (token / cert / OIDC)

```
kubectl config view          # посмотреть (с редактированием секретов)
kubectl config get-contexts  # все контексты
kubectl config use-context <name> # переключиться
kubectl config set-context --current --namespace=prod # привязать NS
```

Imperative vs Declarative

Imperative

```
kubectrl run nginx --image=nginx  
kubectrl scale deploy nginx --replicas=3  
kubectrl edit deploy nginx
```

- Быстро для экспериментов
- Тяжело воспроизвести
- Дрифт от Git

Declarative

```
kubectrl apply -f deployment.yaml
```

- Источник истины — YAML в Git
- Reproducible
- GitOps-friendly
- **Default в production**

Time to make hands dirty — Lab 02

Working with kubectl and kubeconfig

Изучим kubeconfig, контексты, базовые команды `kubectl` .

 **labs/lab-02-kubectl-basics.md**

Part 4: Working with Pods

Атомарная единица развёртывания в Kubernetes

Pod — это shared execution environment

Virtualization → VM. Docker → Container. **Kubernetes** → Pod.

Pod — это разделяемая среда исполнения для **одного или нескольких** контейнеров. Внутри Pod контейнеры разделяют (через Linux namespaces):

- **Network** — общий IP, port range, routing table
- **IPC** — Unix sockets, shared memory
- **UTS** — общий hostname
- **Mount** — общие volumes
- **PID** — общее дерево процессов (опционально)

Атомарность Pod

- Pod — всегда на **одной ноде** (нельзя разнести по узлам)
- Pod либо весь поднят, либо весь не поднят (атомарное развёртывание)
- Pod — минимальная единица **scaling**: добавляем поды, не контейнеры в одном поде
- Все контейнеры в Pod получают **одинаковое имя hostname** (= имя пода)

Pod augments your workload

Pod добавляет:

- **Resource sharing** (см. предыдущий слайд)
- **Advanced scheduling** — nodeSelector, affinity, topology spread
- **Application health probes** — liveness, readiness, startup
- **Restart policies** — Always / OnFailure / Never
- **Security policies** — SecurityContext, capabilities
- **Termination control** — preStop hooks, terminationGracePeriodSeconds
- **Volumes** — emptyDir, configMap, secret, PVC

Pods are mortal and immutable

- **Mortal** — Pod создаётся, живёт, умирает; перезапускать его нельзя
- При сбое K8s создаёт **новый** Pod с новым ID и новым IP
- **Immutable** — нельзя менять Pod после деплоя
- Если нужно изменить — создать новый, удалить старый

✗ "обновить Pod" в K8s означает "удалить и создать новый"

Restart Policies (применяется к контейнерам, не к Pod!)

Policy	Поведение
Always	Перезапустить контейнер при любом завершении (default)
OnFailure	Перезапустить только при ненулевом exit code
Never	Не перезапускать

- Применяется ко **всем** контейнерам в Pod (кроме init containers)
- **Pod сам по себе никогда не рестартится** — kubelet рестартит только контейнеры на той же ноде
- Для перезапуска Pod на другой ноде — нужен controller (Deployment, ReplicaSet)

Static Pods vs Controller-managed Pods

Static Pod

- Создаётся напрямую из манифеста
- Управляется kubelet **на конкретной ноде**
- ❌ Нет self-healing
- ❌ Нет scaling
- ❌ Нет rolling updates
- Используется только для **системных компонентов**

Managed Pod

- Создаётся через Deployment / StatefulSet / DaemonSet
- Управляется controller на control plane
- ✅ Self-healing на других нодах
- ✅ Scaling
- ✅ Rolling updates & rollbacks

99% production workloads = managed pods. Never deploy static pods for apps.

Pod Network

- Каждый Pod получает **свой IP-адрес** в плоской сети
- Любой Pod может общаться с любым Pod в кластере (flat L2 overlay)
- IP назначается через **CNI (Container Network Interface)** плагин
- Популярные CNI: **Cilium** (#1 сейчас), Calico, Flannel
- Внутри Pod контейнеры общаются через `localhost`

Pod CIDR, Service CIDR и Node IP ranges

- **Pod CIDR** — диапазон IP для Pod-ов
- **Service CIDR** — отдельный диапазон для ClusterIP Service-ов
- **Node IPs** — адреса самих VM / физических узлов
- Эти диапазоны **не должны пересекаться**
- Планируйте сеть с запасом по росту, а не "впритык под сегодня"

```
Pods:      10.244.0.0/16
Services:  10.96.0.0/12
Nodes:     192.168.0.0/16
```


CNI — как Pod получает сеть

- **CNI plugin** вызывается kubelet при создании Pod
- Создает **network namespace** для Pod
- Подключает `veth` / bridge / overlay
- Назначает Pod IP и обновляет маршруты
- Популярные реализации: **Calico, Cilium, Flannel**

CNI — это не "одна сеть Kubernetes", а стандарт подключения разных сетевых реализаций.

Multi-container Pod: init containers

Цель: подготовить окружение **до** запуска main app.

- Определяются в `spec.initContainers`
- Запускаются **последовательно, до** обычных контейнеров
- Гарантировано выполняются один раз и завершаются успехом
- Только после этого стартуют app containers

Типичные сценарии:

- Дождаться готовности внешнего сервиса
- Склонировать git-репозиторий
- Прогреть кеш
- Применить миграции БД

Multi-container Pod: sidecar pattern

Цель: добавить функциональность к main app **без изменения** самого app.

Типичные сидкары:

- **Log forwarder** — пишет логи в Kafka / Elasticsearch / Loki
- **Service mesh proxy** — Envoy / Linkerd
- **Configuration syncer** — синкает контент в shared volume
- **Adapter** — конвертирует output главного app

Kubernetes 1.29+

sidecar containers получили **first-class API support**
раньше это был hack через `restartPolicy: Always`
внутри `initContainers`

Pod

Main app
container

Sidecar

Shared

volume

```
<svg viewBox="0 0 520 300" style="position:absolute;
inset:0; width:100%; height:100%; pointer-events:none;">
<defs> <marker id="arrow-sidecar" markerWidth="10"
markerHeight="10" refX="8" refY="3" orient="auto"
markerUnits="strokeWidth"> <path d="M0,0 L0,6 L9,3 z"
fill="#222"></path> </marker> </defs> <line x1="130"
y1="152" x2="224" y2="242" stroke="#222" stroke-
width="2.5" marker-end="url(#arrow-sidecar)"/> <line
x1="288" y1="242" x2="372" y2="152" stroke="#222"
stroke-width="2.5" marker-end="url(#arrow-sidecar)"/>
</svg>
```

Multi-container Pod: adapter pattern

Цель: преобразовать формат данных так, чтобы внешняя система могла их понять без изменения main app.

Когда полезен adapter:

- приложение пишет логи / метрики в "не том" формате
- нужно отдать наружу **Prometheus-friendly**
/metrics
- есть legacy output, который нельзя быстро поменять
- хочется стандартизировать данные на выходе Pod

Идея:

- main app пишет в shared volume или локальный endpoint
- adapter читает данные, **трансформирует** и отдаёт в нужном виде
- monitoring / logging system видит уже

нормализованный формат

Pod

Main app

container

Adapter

Shared

volume

/metrics

```
<svg viewBox="0 0 520 320" style="position:absolute;
inset:0; width:100%; height:100%; pointer-events:none;">
<defs> <marker id="arrow-adapter-top" markerWidth="10"
markerHeight="10" refX="8" refY="3" orient="auto"
markerUnits="strokeWidth"> <path d="M0,0 L0,6 L9,3 z"
fill="#222"></path> </marker> </defs> <polyline
points="124,132 124,196 212,248" fill="none"
stroke="#222" stroke-width="2.5" marker-
end="url(#arrow-adapter-top)"/> <polyline points="304,132
304,202 258,248" fill="none" stroke="#222" stroke-
```

Multi-container Pod: ambassador pattern

Цель: вынести интеграцию с **внешними системами** в отдельный контейнер-посредник.

Когда полезен ambassador:

- **DB / cache proxy** к внешнему кластеру
- **API proxy** к внешнему сервису
- **Shard/router** для выбора нужного backend
- **mTLS / auth wrapper** без изменения main app

Идея:

- main app говорит только с `localhost`
- ambassador знает, **куда** и **как** ходить наружу
- внешний topology / auth / routing скрыт от приложения

Pod

Main app
container

→

localhost

→

Ambassador

app знает только loopback; routing / auth / TLS скрыты
внутри sidecar

Shard 1

Shard 2

Shard n

↗

→

↘

app → `localhost` → ambassador → external shards / API /

DB

Pod manifest — пример

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  labels:
    app: hello
    env: prod
spec:
  containers:
  - name: hello-ctr
    image: nginx:1.27
    ports:
    - containerPort: 80
    resources:
      requests:
        cpu: "500m"
        memory: "256Mi"
      limits:
        cpu: "1"
        memory: "512Mi"
```

минимум, гарантирован

максимум, нельзя превысить

Resource requests vs limits

- **requests** — минимум, который scheduler **гарантирует**
 - Используется при выборе ноды
 - Pod не попадёт на ноду, у которой меньше свободных ресурсов чем requests
- **limits** — максимум, который kubelet **жёстко применяет**
 - CPU: throttling
 - Memory: **OOMKilled** 🧟

Без requests/limits scheduler "слеп". Без limits ваш Pod может съесть всю ноду.

Requests / limits overview

Request

Guaranteed minimum

CPU: **500m**

Memory: **256Mi**

scheduler uses this to pick a node

Container

Running app

request floor

burst zone

hard ceiling

между request и limit контейнер может временно расти

Limit

Maximum allowed

CPU: **1 core**

Memory: **512Mi**

CPU above limit → throttling

Memory above limit → OOMKilled

QoS classes of Pods

BestEffort

Как получить:

вообще не задать `requests` и `limits`

Поведение:

самый низкий приоритет при давлении на ноду

Типичный кейс:

вспомогательные утилиты, временные batch-нагрузки

Node under pressure

BestEffort → Burstable → Guaranteed

Порядок вытеснения (eviction order)

Burstable

Как получить:

есть `requests`, а `limits` выше

или заданы не для всех ресурсов

Поведение:

баланс между гарантией и гибкостью

Типичный кейс:

большинство web/API сервисов

kubectl: introspection commands

```
kubectl get pods                # список (default namespace)
kubectl get pods -A             # из всех namespaces
kubectl get pods -o wide        # + IP, node
kubectl get pod hello-pod -o yaml # полный спек + status

kubectl describe pod hello-pod  # формат + events
kubectl logs hello-pod          # логи 1-го контейнера
kubectl logs hello-pod -c sidecar -f # логи конкретного контейнера, follow
kubectl logs hello-pod --previous # логи предыдущей инстанции

kubectl exec -it hello-pod -- sh # интерактивный shell
kubectl exec hello-pod -- ps     # одна команда

kubectl edit pod hello-pod      # править live (только mutable поля!)
kubectl delete pod hello-pod    # удалить
```

Time to make hands dirty — Lab 03

Deploy your first Pod

Декларативно поднимаем простой Pod, делаем introspection.

 **labs/lab-03-first-pod.md**

Time to make hands dirty — Lab 04

Multi-container Pod (init + sidecar)

Init container ждёт Service, sidecar синкает контент.

 **labs/lab-04-multi-container-pod.md**

Part 5: Namespaces

Виртуальные кластеры внутри одного физического

Namespace ≠ kernel namespace

- **Kernel namespaces** — изолируют процессы (контейнеры)
- **Kubernetes Namespaces** — делят кластер на виртуальные кластеры

⚠ Namespaces — это **soft isolation**, не security boundary!

Compromised workload в одном namespace **может** повлиять на другой.

Default Namespaces в каждом кластере

Namespace	Назначение
default	Куда попадают объекты без <code>-n</code>
kube-system	Control plane components (CoreDNS, kube-proxy, scheduler...)
kube-public	Объекты, читаемые всеми (включая неавторизованных)
kube-node-lease	Heartbeat'ы нод

```
kubectl get namespaces      # или: kubectl get ns
kubectl get pods -n kube-system # посмотреть control plane
kubectl get pods --all-namespaces # = -A
```

Namespaced vs cluster-scoped resources

```
kubectl api-resources --namespaced=true # резиденты namespace  
kubectl api-resources --namespaced=false # cluster-wide
```

Namespaced

- Pod, Deployment, Service
- ConfigMap, Secret
- PVC, Role, RoleBinding
- Ingress, NetworkPolicy

Cluster-scoped

- Node
- PersistentVolume
- StorageClass
- ClusterRole, ClusterRoleBinding
- Namespace (сам по себе)
- CRD

Создание и работа с Namespaces

```
# Imperative
kubectl create ns dev

# Declarative
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    env: production
EOF

# Привязать kubectl к namespace
kubectl config set-context --current --namespace=dev

# Удалить (! каскадно удалит ВСЁ в namespace)
kubectl delete ns dev
```

Time to make hands dirty — Lab 05

Working with Namespaces

Создадим namespaces, развернём приложение в выделенный namespace, посмотрим на изоляцию.

 **labs/lab-05-namespaces.md**

Part 6: Deployments

Self-healing, scaling, rolling updates, rollbacks

Why Deployments?

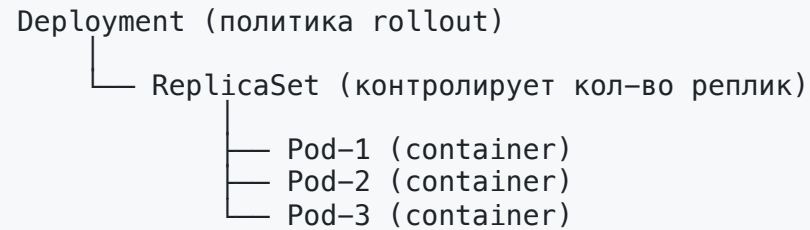
Pods are mortal. Static pods can't:

- ❌ Self-heal across nodes
- ❌ Scale
- ❌ Roll out updates
- ❌ Roll back

Deployment обёртывает Pod и добавляет:

- ✅ Self-healing
- ✅ Manual + auto scaling
- ✅ Zero-downtime rolling updates
- ✅ Versioned rollbacks

Архитектура: Deployment → ReplicaSet → Pod



- **Deployment** — определяет desired state и стратегию обновления
- **ReplicaSet** — поддерживает заданное число реплик
- **Pod** — атомарная единица исполнения

Всегда работаем с **Deployment**, не с ReplicaSet или Pod напрямую.

Deployment manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 10
  revisionHistoryLimit: 5           # сколько старых RS хранить
  progressDeadlineSeconds: 300     # дать новой реплике 5 мин стартовать
  minReadySeconds: 10              # пауза между заменами
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1            # минимум доступных = N-1
      maxSurge: 1                  # максимум = N+1
  selector:
    matchLabels:
      app: hello-world
  template:                         # это шаблон Pod-a
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-ctr
          image: nginx:1.27
          ports: [{containerPort: 80}]
```

Rolling Update механика

`maxSurge: 1` + `maxUnavailable: 1` при `replicas: 10`:

- Минимум доступных: **9**
- Максимум всего: **11**
- За шаг обновляется **2 Pod**

```
# Применить изменения (например, новый image tag)
kubectl apply -f deploy.yaml
```

```
# Наблюдать
kubectl rollout status deployment hello-deploy
```

```
# Пауза / возобновление
kubectl rollout pause deployment hello-deploy
kubectl rollout resume deployment hello-deploy
```

Rollback к прошлой версии

```
# История ревизий
kubectl rollout history deployment hello-deploy

# Подробнее про конкретную ревизию
kubectl rollout history deployment hello-deploy --revision=2

# Откатить к предыдущей
kubectl rollout undo deployment hello-deploy

# Откатить к конкретной
kubectl rollout undo deployment hello-deploy --to-revision=1
```

⚠ После rollback **синхронизируйте YAML** в Git — иначе следующий apply перетрёт откат.

Autoscaling — preview (детально в Part 16)

- **HPA (Horizontal Pod Autoscaler)** — добавляет/убирает Pod-ы по метрикам
- **VPA (Vertical Pod Autoscaler)** — меняет CPU/memory у существующих Pod-ов
- **Cluster Autoscaler (CA)** — добавляет/удаляет ноды
- **KEDA** — event-driven autoscaling (Kafka, RabbitMQ, custom metrics)

Multi-dimensional autoscaling = комбинация: HPA добавляет Pod-ы → CA добавляет ноды.

Time to make hands dirty — Lab 06

Deployment — create, scale, expose

Развёртываем 10 реплик, скейлим вверх/вниз, экспонируем через LoadBalancer.

 **labs/lab-06-deployment-basics.md**

Time to make hands dirty — Lab 07

Rolling Update & Rollback

Обновляем image, наблюдаем rolling update, откатываемся к предыдущей версии.

 **labs/lab-07-rolling-update-rollback.md**

Part 7: Services

Стабильное сетевое имя для пула Pod-ов

Why Services?

Pod-ы умирают и появляются с **новыми IP**:

- Scale up / down → новые IP
- Rolling updates → новые IP
- Failures → новые IP

→ Клиенты не могут полагаться на IP Pod-ов.

Service даёт:

-  Стабильное DNS-имя
-  Стабильный ClusterIP
-  Round-robin load balancing
-  Только к healthy Pod-ам (через probes)

EndpointSlice — список здоровых Pod-ов

При создании Service автоматически создаётся **EndpointSlice**:

- Список Pod IP'ов, на которые слать трафик
- Обновляется автоматически когда Pod'ы появляются/исчезают
- Только **healthy** Pod-ы (прошедшие readinessProbe)
- Если >100 Pod-ов — создаётся несколько EndpointSlice'ов (для производительности)

```
kubectl get endpointslices  
kubectl describe endpointslice <name>
```

До K8s 1.21 использовались **Endpoints** объекты — функционально аналогичные, но менее производительные.

Service types

ClusterIP (default)

- IP только внутри кластера
- DNS-имя зарегистрировано в CoreDNS
- Не доступен снаружи
- Большинство внутренних сервисов

NodePort

- Открывает порт **30000-32767** на каждой ноде
- Извне: `<NodeIP>:<NodePort>`
- Строится поверх ClusterIP

LoadBalancer

- Создает **облачный LB** автоматически
- Получает публичный IP
- Только в cloud-кластерах
- Строится поверх NodePort

ExternalName

- Просто DNS CNAME alias
- Никаких Pod-ов сзади
- Полезно для интеграции с внешними системами

Labels и selectors — как Service находит Pod-ы

```
apiVersion: v1
kind: Service
metadata:
  name: app-svc
spec:
  selector:
    app: my-app
    environment: production
```

- **Labels** — произвольные key/value метки на объектах
- **Selectors** — фильтр, который выбирает объекты по этим меткам
- Service, Deployment и NetworkPolicy опираются именно на labels, а не на имена Pod-ов

ClusterIP Service example

```
apiVersion: v1
kind: Service
metadata:
  name: app-svc
spec:
  type: ClusterIP          # default; можно опустить
  ports:
    - port: 80             # порт самого Service
      targetPort: 8080     # порт в контейнере
      protocol: TCP
  selector:
    app: hello-world       # к каким Pod-ам слать
```

После создания доступен изнутри кластера по:

- DNS: `app-svc`, `app-svc.default.svc.cluster.local`
- IP: `kubectl get svc app-svc` → CLUSTER-IP

LoadBalancer Service

```
apiVersion: v1
kind: Service
metadata:
  name: app-lb
spec:
  type: LoadBalancer
  ports:
  - port: 80
    targetPort: 8080
  selector:
    app: hello-world
```

Что произойдёт:

1. K8s создаст Service
2. Cloud-controller-manager создаст **облачный LB** (AWS ALB / GCP LB / Azure LB)
3. LB получит публичный IP
4. Трафик: Client → Cloud LB → NodePort → ClusterIP → Pod

Time to make hands dirty — Lab 08

Service Types — ClusterIP, NodePort, LoadBalancer

Разберём практически все три типа Service. Подключимся к Pod-ам через каждый.

 **labs/lab-08-services.md**

Part 8: Ingress

Один LoadBalancer — много приложений

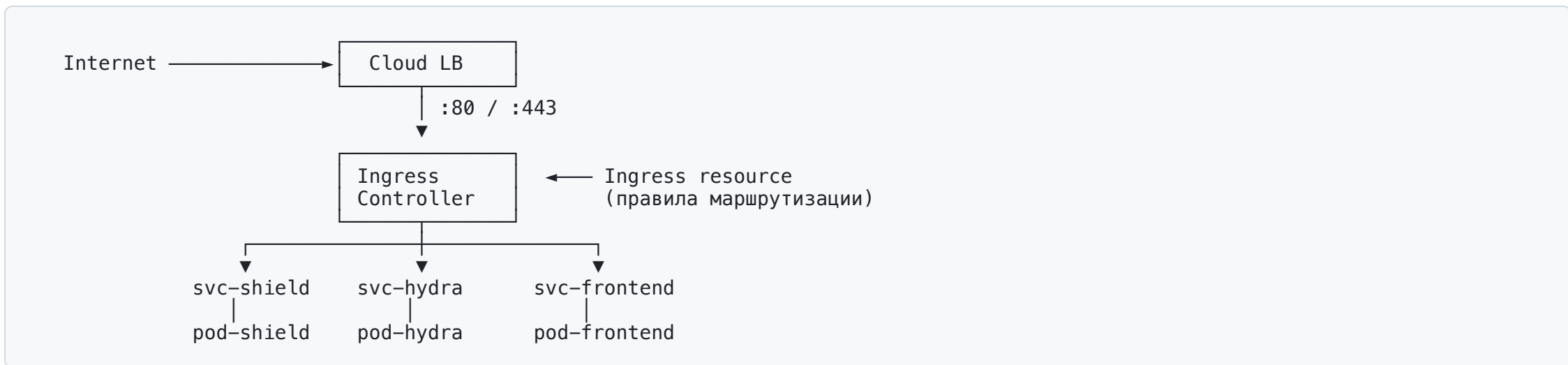
Problem: LoadBalancer per app не масштабируется

- 25 app → 25 cloud LB → 💰💰💰
- Каждый LB имеет свой IP, свой биллинг
- Сложно управлять DNS

Ingress — единая точка входа:

- Один cloud LB / один публичный IP
- Маршрутизация по **host** (mydomain.com → app-a)
- Маршрутизация по **path** (api.com/users → users-svc, api.com/orders → orders-svc)
- TLS termination
- L7 (HTTP/HTTPS)

Ingress architecture



Два компонента:

1. **Ingress resource** — описывает правила (где какой Service)
2. **Ingress controller** — реализует правила (NGINX, Traefik, etc.)

Ingress resource — пример

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mcu-all
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    # Host-based
    - host: shield.mcu.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: svc-shield
                port: {number: 8080}
    # Path-based
    - host: mcu.com
      http:
        paths:
          - path: /shield
            pathType: Prefix
            backend:
              service:
                name: svc-shield
                port: {number: 8080}
          - path: /hydra
            pathType: Prefix
            backend:
              service:
                name: svc-hydra
                port: {number: 8080}
```

Gateway API — разделение ролей и маршрутов

Цепочка объектов

- **GatewayClass** → реализация контроллера
- **Gateway** → listeners, ports, TLS
- **HTTPRoute** → host, path, headers, weight
- **Service** → backend для Pod-ов
- Поддерживает не только HTTP: TCP, UDP, gRPC, TLS passthrough
- GA в K8s 1.30 (2024)

Кто чем управляет

- **Infra / provider** → GatewayClass + controller
- **Platform team** → Gateway
- **App team** → HTTPRoute
- **Cross-namespace access** → ReferenceGrant

⚠ Ingress NGINX retirement March 2026 → миграция на Gateway API становится мейнстримом.

Gateway API model

GatewayClass

implementation type

Envoy Gateway, Cilium, Istio, NGINX Gateway Fabric

→

Gateway

listeners

ports / hostnames / TLS

→

Routes

HTTPRoute

GRPCRoute / TCPRoute / UDPRoute

matches + filters + weights

→

Service backend

Service → Pods

final hop inside cluster

Infrastructure

GatewayClass and controller decide how the proxy is implemented

Gateway API workflow

Infra / provider

GatewayClass

Gateway controller

defines implementation

→

Platform team

Gateway

listeners / ports / TLS

publishes shared endpoint

→

App team

HTTPRoute / GRPCRoute

binds to Service backend

owns traffic rules

Controller loop

watches GatewayClass, Gateway and Route objects in the API Server

→

Traffic path

Time to make hands dirty — Lab 09

Ingress — host and path routing

Поставим NGINX Ingress Controller, развернём 2 app, маршрутизируем по host и path.

 **labs/lab-09-ingress.md**

Part 9: Service Discovery

Как поды находят друг друга через DNS

Два аспекта service discovery

1. **Registration** — регистрация Service в реестре
2. **Discovery** — поиск Service по имени → IP

В Kubernetes оба — **автоматические**:

- Service зарегистрирован в **cluster DNS** сразу при создании
- Pod-ы автоматически настроены резолвить через cluster DNS

Cluster DNS = CoreDNS

- В каждом кластере K8s крутится **CoreDNS** (раньше — kube-dns)
- Развёрнут как Deployment в `kube-system` namespace
- Service-объект `kube-dns` указывает на CoreDNS Pod-ы
- ClusterIP `kube-dns` Service — это **тот же IP**, который пишется в `/etc/resolv.conf` каждого Pod-а

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl get svc -n kube-system kube-dns
# CLUSTER-IP обычно 10.96.0.10 или подобное
```

/etc/resolv.conf внутри Pod

```
kubectl exec my-pod -- cat /etc/resolv.conf
```

```
search default.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
```

- **nameserver** — IP CoreDNS Service
- **search** — search domains, добавляются к short names
- **ndots:5** — если в имени <5 точек, K8s сначала пробует search domains

Pod автоматически получает search domain своего namespace.

FQDN формат Service

```
<service-name>.<namespace>.svc.cluster.local
```

Примеры:

- `kube-dns.kube-system.svc.cluster.local` — CoreDNS
- `app-svc.default.svc.cluster.local` — App Service в default
- `mongo.prod.svc.cluster.local` — MongoDB Service в prod

Внутри **того же namespace** можно использовать короткое имя:

- `app-svc` ✓

Из другого namespace — нужно FQDN или хотя бы `<svc>.<namespace>` :

- `app-svc.prod` ✓ (благодаря search domains)

kube-proxy — невидимый герой

ClusterIP — это **виртуальный** IP. Никакого устройства с этим IP не существует.

Как трафик доходит до Pod-а:

1. App резолвит имя через CoreDNS → ClusterIP
2. App шлёт пакет на ClusterIP
3. **kube-proxy** программирует kernel rules (iptables / IPVS / eBPF) на каждой ноде
4. Пакет перехватывается и редиректится на IP **healthy Pod-а** (из EndpointSlice)

```
# Посмотреть iptables rules от kube-proxy (на ноде)
sudo iptables -t nat -L KUBE-SERVICES | head -20
```

Time to make hands dirty — Lab 10

Service Discovery & DNS

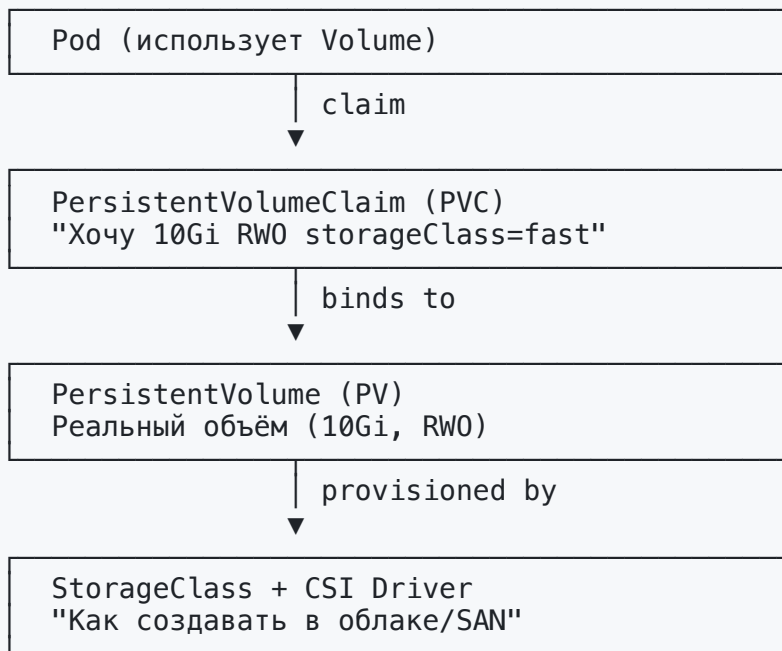
Развернём 2 app в 2 namespace, проверим short names и FQDN, поломаем DNS и починим.

 **labs/lab-10-service-discovery.md**

Part 10: Storage

PV, PVC, StorageClass и CSI

Архитектура storage в K8s



Volume vs PersistentVolume

Volume

Привязан к жизненному циклу Pod-а.

- `emptyDir` — временное
- `hostPath` — путь на ноде
- `configMap`, `secret` — инъект конфигов
- `nfs` — простая NFS-маунт

PersistentVolume (PV)

Переживает Pod.

- `awsElasticBlockStore`
- `azureDisk`, `gcePersistentDisk`
- `csi` — generic CSI driver
- Cluster-scoped ресурс
- Создаётся вручную ИЛИ автоматически (dynamic provisioning)

StorageClass — шаблон провижининга

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: ebs.csi.aws.com          # ← CSI driver
parameters:
  type: gp3                          # SSD
  encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete                # или Retain
allowVolumeExpansion: true
```

StorageClass-ы **immutable** после создания. Хотите изменить — создавайте новый.

Access modes

Mode	Описание
ReadWriteOnce (RWO)	Read/write одной нодой, несколько Pod-ов на ней
ReadOnlyMany (ROX)	Read-only с нескольких нод
ReadWriteMany (RWX)	Read/write с нескольких нод
ReadWriteOncePod (RWOP)	Read/write одним Pod-ом (K8s 1.27+)

- **Block storage** (EBS, Persistent Disk) — обычно только **RWO**
- **File storage** (NFS, EFS, GlusterFS, CephFS) — поддерживают **RWX**

Reclaim policy

Что делать с PV когда PVC удалён:

- **Delete** (default для dynamic provisioning) — удалить и PV, и облачный диск 💣
- **Retain** — оставить PV, диск надо чистить вручную

⚠ Production databases: **только Retain**. Иначе один accident `kubectl delete pvc` → потеря данных.

PVC + Pod пример

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: fast
  resources:
    requests:
      storage: 10Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - mountPath: /data
          name: data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: my-pvc
```

Time to make hands dirty — Lab 11

PV / PVC / StorageClass — dynamic provisioning

Создадим StorageClass, PVC, и Pod который использует динамически выделенный volume.

 **labs/lab-11-storage.md**

Part 11: ConfigMaps & Secrets

Декаплинг приложения от конфигурации

Why ConfigMaps?

Антипаттерн: запекать конфиг в Docker-образ.

- ❌ Один образ для dev / test / prod → разные сборки
- ❌ Smallest config change = full rebuild + redeploy
- ❌ Нельзя переиспользовать образ

ConfigMap — храним конфиг отдельно:

- ✅ Один образ, разная конфигурация per environment
- ✅ Изменения конфига без rebuild
- ✅ Tracked in Git (отдельно от image)

ConfigMap — k-v storage

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database_url: "postgres://db:5432/app"
  log_level: "info"
  app.properties: |
    timeout=30
    retries=5
    feature.flags=v2
```

- Keys: буквы, цифры, точки, дефисы, подчёркивания
- Values: любой текст (включая многострочные файлы)
- Total size: **до 1 MiB**

 ConfigMap — **не для секретов!** Не хранит пароли, ключи, токены. Для этого — Secrets.

Три способа использовать ConfigMap в Pod

1. Environment variables

```
spec:
  containers:
  - name: app
    env:
    - name: DB_URL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: database_url
```

2. Command arguments

```
command: ["echo"]
args: ["$(LOG_LEVEL)"]
env:
- name: LOG_LEVEL
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: log_level
```

3. Volume mount (📌 рекомендуется)

```
volumes:
- name: config
  configMap:
    name: app-config
containers:
- volumeMounts:
  - mountPath: /etc/app
    name: config
```

Secrets

Все то же самое, что ConfigMap, но:

- Данные **base64-encoded** (не encrypted!)
- Монтируются как **tmpfs** (в памяти, не на диске узла)
- Доступ контролируется через **RBAC** более строго
- Поддерживают разные **типы**:
 - `opaque` — generic
 - `kubernetes.io/tls` — TLS сертификаты
 - `kubernetes.io/dockerconfigjson` — Docker registry credentials
 - `kubernetes.io/ssh-auth` — SSH ключи

Secret manifest

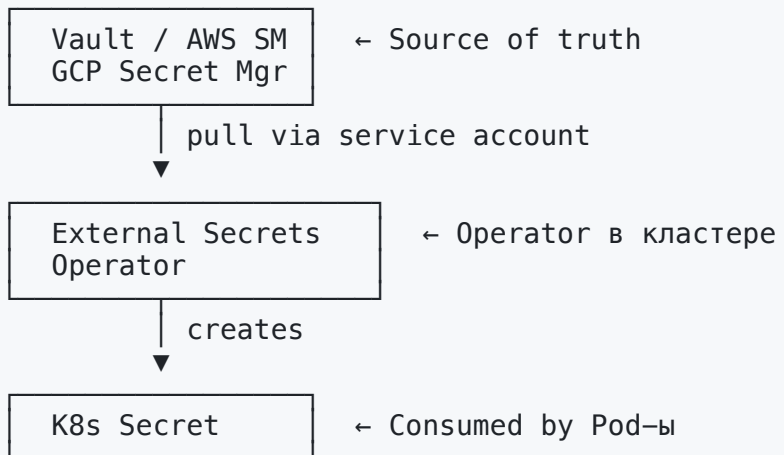
```
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
data:
  username: YWRtaW4=          # base64
  password: cGFzc3dvcmQxMjM= # base64
```

Или через `stringData` (K8s сам закодирует в base64):

```
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
stringData:
  username: admin
  password: password123
```

Использовать в Pod — так же, как ConfigMap (env / volumes).

External secrets — production approach



External Secrets overview

External secret store

Vault

AWS Secrets Manager

GCP Secret Manager

→

External Secrets Operator

reads via SecretStore / ClusterSecretStore

syncs on interval / change

→

Kubernetes Secret

created / refreshed by ESO

consumed by Pods as env / volume

Git stores

ExternalSecret manifest, not raw password

Rotation

secret backend updates can flow into K8s automatically

RBAC blast radius

limit who can read resulting K8s Secret

Time to make hands dirty — Lab 12

ConfigMaps — все три способа использования

Env vars, command args, volume mount. Понаблюдаем как обновляется только volume.

 **labs/lab-12-configmaps.md**

Time to make hands dirty — Lab 13

Secrets — create, mount, decode

Создаём Secret, монтируем как volume, проверяем что внутри Pod-а файлы — plaintext.

 **labs/lab-13-secrets.md**

Part 12: StatefulSets

Persistent identity и storage для stateful приложений

StatefulSet vs Deployment

Aspect	Deployment	StatefulSet
Pod names	Random suffix (app-7f8d9-xz9k)	Ordered (app-0 , app-1 , app-2)
Pod startup	Параллельно	По одному, ждёт ready
Pod shutdown	Параллельно	В обратном порядке
DNS	Только Service	+ DNS per Pod
Storage	Shared / per Pod	Sticky volumes per Pod
Use case	Stateless apps	Databases, queues

StatefulSet manifest

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx-headless"      # ← Headless Service для peer discovery
  replicas: 3
  selector:
    matchLabels: {app: nginx}
  template:
    metadata:
      labels: {app: nginx}
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          volumeMounts:
            - name: www
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:              # ← один PVC per Pod
    - metadata:
        name: www
      spec:
        accessModes: [ReadWriteOnce]
        storageClassName: fast
        resources: {requests: {storage: 1Gi}}
```

Headless Service

Для StatefulSet нужен **headless Service** (`clusterIP: None`):

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-headless
spec:
  clusterIP: None      # ← Headless
  selector:
    app: nginx
  ports:
    - port: 80
```

Что это даёт:

- **DNS per Pod:** `web-0.nginx-headless.default.svc.cluster.local`
- DNS A-record возвращает **все Pod IPs** (для peer discovery)
- HE load-balancing — это просто DNS registry

Когда использовать StatefulSet

✗ Плохой use-case

- Stateless web / API → лучше Deployment
- Batch-задачи → лучше Job / CronJob
- Managed DB через operator → operator сам создаст StatefulSet

✓ Хороший use-case

- Self-hosted databases без operator
- Caches и brokers с кластеризацией
- Distributed systems со stable identity
- Apps, которым нужен ordered startup / shutdown

Time to make hands dirty — Lab 14

StatefulSet with persistent storage

Развернём StatefulSet с volumeClaimTemplates, проверим sticky volumes, DNS peer discovery.

 **labs/lab-14-statefulset.md**

Part 13: API Security & RBAC

Authentication, Authorization, Admission

Жизненный цикл API request

```
Client → TLS → AuthN → AuthZ → Admission → etcd
  kubectl  certs  RBAC    mutating
                        validating
```

1. **Transport security** — TLS, mutual auth
2. **Authentication (AuthN)** — кто вы?
3. **Authorization (AuthZ)** — что вам можно делать?
4. **Admission control** — соответствует ли политикам кластера?
5. **Persist** — записать в etcd

Любой шаг может отвергнуть request. Failed AuthN → 401. Failed AuthZ → 403.

Authorization — RBAC

Kubernetes RBAC API: 4 объекта

Role (namespaced)

Список разрешённых действий в **одном namespace**.

ClusterRole (cluster-scoped)

Список действий **во всём кластере** ИЛИ
переиспользуемый шаблон для нескольких namespace.

RBAC — **deny by default**, только allow rules (нет deny).

RoleBinding

Связывает **Role** (или ClusterRole) с user/group/SA в
конкретном namespace.

ClusterRoleBinding

Связывает **ClusterRole** с user/group/SA на **уровне кластера**.

Role + RoleBinding example

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: ["" ]           # core API group
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

ServiceAccount — identity для Pod-ов

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: monitoring-sa
  namespace: monitoring
---
apiVersion: v1
kind: Pod
spec:
  serviceAccountName: monitoring-sa    # ← Pod использует этот SA
  containers:
  - name: prom
    image: prom/prometheus
```

- Каждый Pod использует SA для аутентификации в Kube API
- Token автоматически монтируется в `/var/run/secrets/kubernetes.io/serviceaccount/token`
- Если Pod НЕ должен ходить в API — отключите: `automountServiceAccountToken: false`

Best practices — RBAC

- ✓ **Principle of least privilege** — давайте только то, что нужно
- ✓ Используйте **Group-based bindings** (не user-by-user)
- ✓ Разделяйте права по **namespace** для команд
- ✓ Проводите аудит RBAC регулярно — `kubectl auth can-i`
- ✗ Не используйте `cluster-admin` для повседневной работы
- ✗ Не используйте wildcards (`*`) кроме как для admin tools

```
# Проверить права
kubectl auth can-i create pods --as alice -n dev
kubectl auth can-i '*' '*' --as system:masters
```


Admission Controllers — policy после AuthZ

Два типа:

- **Mutating** — могут менять request (например, добавить sidecar, set defaults)
- **Validating** — могут только approve/reject

Built-in controllers:

- `NodeRestriction` — kubelet может менять только свои Pod-ы
- `PodSecurity` — Pod Security Standards (privileged / baseline / restricted)
- `AlwaysPullImages` — всегда тянуть образ

Кастомные через webhook:

- **OPA Gatekeeper, Kyverno, Kubewarden**

Time to make hands dirty — Lab 15

RBAC + Service Account

Создадим SA, Role, RoleBinding. Запустим Pod с SA и проверим can-i.

 **labs/lab-15-rbac.md**

Part 14: The Kubernetes API & CRDs

Расширяем Kubernetes своими ресурсами

Kubernetes — это REST API + контроллеры

Kubernetes API as the control hub

Clients

kubectl, curl, CI/CD, SDKs

CRUD over HTTPS

Control plane

API Server

AuthN / AuthZ / Admission

etcd

desired state store

Scheduler

chooses a node

Controller manager

reconciliation loops

Clients → API Server → etcd

Scheduler and controllers also talk through API Server

Data plane node

kubelet

kube-proxy

Pods

API Server = единая точка входа

Всё, что вы создаёте — это просто HTTP-запросы к **API Server**:

K8s verb	HTTP method	kubectl
create	POST	<code>kubectl create -f</code>
get / list	GET	<code>kubectl get</code>
update	PUT	(через apply/edit)
patch	PATCH	<code>kubectl patch</code>
delete	DELETE	<code>kubectl delete</code>

Ключевая идея:

- **kubectl** не говорит напрямую с Pod-ами
- **scheduler** и **controllers** тоже ходят через API Server
- **etcd** не трогают напрямую прикладные клиенты
- Kubernetes = **API + controllers + reconciliation**

API groups

```
/api/v1                                ← core group (старые ресурсы)
  pods, services, secrets, namespaces

/apis/apps/v1                          ← named group
  deployments, replicaset, statefulsets, daemonsets

/apis/networking.k8s.io/v1
  ingresses, networkpolicies, gatewayclasses

/apis/rbac.authorization.k8s.io/v1
  roles, rolebindings, clusterroles, clusterrolebindings

/apis/storage.k8s.io/v1
  storageclasses

/apis/batch/v1
  jobs, cronjobs
```

GVR = Group + Version + Resource — однозначно идентифицирует тип объекта

Lifecycle: alpha → beta → stable

```
v1alpha1 → экспериментальное, может сломаться  
v1beta1  → pre-release, можно использовать в тестах  
v1       → GA, production-ready
```

- Alpha API — disabled by default, нужно включить feature flag
- Beta API — enabled by default
- GA (stable) — гарантия поддержки 1 год / 3 релиза после deprecation

Пример deprecation warning:

```
Warning: extensions/v1beta1 Ingress is deprecated in v1.14+,  
         unavailable in v1.22+; use networking.k8s.io/v1 Ingress
```

Custom Resource Definitions (CRD)

K8s можно **расширить** своими типами:

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.mycorp.com
spec:
  group: mycorp.com
  scope: Namespaced
  names:
    plural: backups
    singular: backup
    kind: Backup
    shortNames: [bk]
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                schedule: {type: string}
                retention: {type: integer}
```


Operator pattern

Custom Resource (CR)

```
apiVersion: mycorp.com/v1
kind: Backup
spec:
  schedule: "0 2 * * *"
  retention: 7
```

Controller / Operator

←

```
Watch all "Backup" CRs
When new Backup found:
  - run pg_dump
  - upload to S3
  - update status
When schedule fires:
  - same logic
Update spec → reconcile
```

Известные Operators:

- **CloudNativePG** — PostgreSQL operator
- **Prometheus Operator** — управляет Prometheus стэком
- **Strimzi** — Kafka operator
- **Argo CD** — GitOps via CRD
- **Cert-Manager** — Issuer, Certificate, ClusterIssuer CRDs

Time to make hands dirty — Lab 16

Explore K8s API + define a CRD

Через kubectl проху "потрогаем" REST API. Затем создадим свой CRD и custom object.

 **labs/lab-16-api-crd.md**

Part 15: Threat Modeling & Real-world Security

STRIDE, SecurityContext, Pod Security, NetworkPolicy

STRIDE — фреймворк threat modeling

Threat	В K8s контексте
Spoofing	Притвориться другим Pod-ом, нодой, юзером
Tampering	Изменить образы, манифесты, ConfigMaps
Repudiation	Отрицание действий (без audit log нельзя доказать)
Information disclosure	Утечка Secrets, etcd, PV-данных
Denial of service	Перегрузить API server, node
Elevation of privilege	Контейнер → нода, namespace → cluster

Идея: для каждой angle подумать "какие у нас controls и где gaps?"

Pod Security Standards (PSS)

3 уровня:

- **privileged** — без ограничений (для системных Pod-ов)
- **baseline** — запрещает наиболее опасные настройки (running as root, hostPath и др.)
- **restricted** — максимально строгий (must run as non-root, drop ALL capabilities, etc.)

3 режима применения:

- **enforce** — нарушающие Pod-ы отклоняются
- **audit** — логируется в audit log
- **warn** — warning в kubectl

Применяются через **labels** на **namespace**:

```
kubectl label ns my-ns \
  pod-security.kubernetes.io/enforce=baseline \
  pod-security.kubernetes.io/warn=restricted
```

SecurityContext — настройки безопасности Pod/Container

```
apiVersion: v1
kind: Pod
spec:
  securityContext:                                # Pod-level
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: app
    image: my-app:1.0
    securityContext:                              # Container-level (overrides Pod)
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
        add: ["NET_BIND_SERVICE"]
```

runAsNonRoot и runAsUser: безопасный запуск начинается с образа

- **runAsNonRoot: true** — базовая защита от запуска процесса как root
- **runAsUser / runAsGroup** — фиксируют конкретные UID/GID
- Если образ рассчитан только на root, Kubernetes-флаг сам по себе не спасёт
- Лучше, когда non-root пользователь задан ещё в **Dockerfile**

```
securityContext:  
  runAsNonRoot: true  
  runAsUser: 10001  
  runAsGroup: 10001
```

Файловая система и kernel guardrails

- **readOnlyRootFilesystem** — root FS только для чтения
- **fsGroup** — помогает согласовать права на mounted volumes
- **procMount: Unmasked** — только для очень специальных сценариев
- **seLinuxOptions / sysctls** — тонкая настройка, обычно только с участием platform team

Чем меньше контейнер видит и может менять, тем меньше поверхность атаки.

Linux Capabilities в K8s

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]    # биндить port < 1024
```

Capability	Что даёт
NET_RAW	Raw sockets (ping) — атаки уровня L2
NET_ADMIN	Manage network interfaces
SYS_ADMIN	Практически root внутри cgroups/namespaces
CHOWN	Менять owner файлов
KILL	Слать сигналы процессам

Default capabilities (containerd, CRI-O) включают многое. Лучше: `drop: ["ALL"]` + `add` точно.

Network Policies — firewall для Pod-ов

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-allow-frontend
  namespace: prod
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - port: 8080
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: db
      ports:
        - port: 5432
```

Default — **разрешено всё**. NetworkPolicy переключают на **deny-by-default** для отобранных Pod-ов.

Default-deny pattern

```
# 1. Default deny all
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: prod
spec:
  podSelector: {} # все Pod-ы в namespace
  policyTypes: [Ingress, Egress]
---
# 2. Allow DNS (без этого Pod-ы сломаются)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: prod
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector:
        matchLabels: {kubernetes.io/metadata.name: kube-system}
    ports:
    - port: 53
      protocol: UDP
```

Image security в pipeline

- ✓ **Use approved base images** — small, hardened
- ✓ **Scan для CVE** — Trivy, Snyk, Gype в CI
- ✓ **Sign images** — Cosign / Sigstore
- ✓ **Private registry** — не Docker Hub для production
- ✓ **Image pull policy: Always** в prod (свежий pull, не cached)
- ✓ **Verify signature через admission** — Sigstore policy controller / Kyverno

```
# Scan image
trivy image my-app:1.0

# Sign with Cosign
cosign sign --key cosign.key my-registry.com/app:1.0

# Verify
cosign verify --key cosign.pub my-registry.com/app:1.0
```

Hard vs soft multi-tenancy

Aspect	Soft	Hard
Tenants	Trusted (внутри org)	Untrusted (different customers)
Isolation	Namespaces	Separate clusters
Examples	dev/test/prod, teams	SaaS customers
Cost	Low	High

 Namespaces — **NE** hard isolation boundary. Враждебные нагрузки требуют отдельных кластеров.

Virtual Clusters (vcluster)

- Один физический кластер хостит **несколько виртуальных control plane**
- У каждого tenant-а свои namespace, API-объекты, RBAC и often отдельный kubeconfig
- Меньше стоимость, чем отдельный cluster per tenant
- Сильнее изоляция и автономность, чем просто namespaces

Хороший компромисс между soft multi-tenancy и отдельным cluster на каждого tenant-а.

Time to make hands dirty — Lab 17

SecurityContext + Pod Security Admission

Hardened Pod c runAsNonRoot, readOnlyRootFilesystem, dropped capabilities + apply baseline PSA.

 **labs/lab-17-securitycontext-psa.md**

Time to make hands dirty — Lab 18

NetworkPolicy — micro-segmentation

Default-deny + explicit allow для frontend → backend → db.

 **labs/lab-18-networkpolicy.md**

Part 16: Scheduling, Scaling & Batch

Scheduling · Autoscaling · DaemonSet · Job · CronJob

Scheduler — как Pod находит ноду

Две фазы:

1. Filtering — отсеять неподходящие ноды

- хватает ресурсов?
- проходят `taints/tolerations` ?
- подходит `nodeSelector / affinity` ?

2. Scoring — оценить оставшиеся

- меньше текущая нагрузка?
- image уже cached?
- лучше spread по zones / nodes?

Победитель → **Pod assigned**.

Если после filtering не осталось кандидатов, Pod уходит в **Pending**, а дальше может подключиться **Cluster Autoscaler**.

Decision flow

Pending Pod
needs a node

1. Filtering

Node A

pass

Node B

fail: taint

Node C

fail: CPU

2. Scoring

Node A

78 points

Node D

92 points

Winner: Node D

Pod assigned to node

<svg viewBox="0 0 920 380" style="position:absolute;

nodeSelector — самый простой способ

```
spec:
  nodeSelector:
    disktype: ssd
    zone: us-east-1a
```

Label ноды:

```
kubectl label node node-1 disktype=ssd
kubectl label node node-1 zone=us-east-1a
kubectl get nodes --show-labels
```

Что важно:

- `nodeSelector` — это **точное совпадение label-ов**
- все ключи работают как **AND**
- если совпадений нет, Pod останется **Pending**

Как scheduler принимает решение

Pod request

my-app Pod

disktype=ssd

zone=us-east-1a

Нужны обе метки сразу

Node A 

disktype=ssd

zone=us-east-1a

Pod can be scheduled here

Node B 

disktype=hdd

zone=us-east-1a

One label mismatched → not eligible

Node Affinity — гибче nodeSelector

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:    # hard rule
        nodeSelectorTerms:
          - matchExpressions:
              - key: instance-type
                operator: In
                values: [m5.large, m5.xlarge]
      preferredDuringSchedulingIgnoredDuringExecution:    # soft preference
        - weight: 70
          preference:
            matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: [us-east-1a]
```

operators: In, NotIn, Exists, DoesNotExist, Gt, Lt

Pod Affinity / Anti-Affinity

Pod Affinity — "посади меня рядом" с Pod-ами по label selector.

Подходит для:

- app + cache / app + proxy рядом на одной ноде
- data locality
- снижение network latency между тесно связанными сервисами

Pod Anti-Affinity — "не сажай меня рядом" с такими Pod-ами.

Подходит для:

- HA-реплик одного Deployment
- уменьшения blast radius
- распределения по нодам / зонам

Affinity

Node A

Pod A

app=web

New Pod

wants nearby

Node B

No matching peer

Anti-Affinity

Node A

Replica 1

app=api

Replica 2

avoid same node

Node B

Replica 2 scheduled here

Taints and Tolerations

Taint — ограничение на ноде:

“не размещай сюда Pod-ы без допуска”

```
kubectl taint node node-1 gpu=true:NoSchedule
```

Toleration — разрешение в Pod:

“этому Pod можно игнорировать такой taint”

```
список:
  tolerations:
    - key: gpu
      operator: Equal
      value: "true"
      effect: NoSchedule
```

Эффекты:

- **NoSchedule** — новые Pod-ы не ставим
- **PreferNoSchedule** — по возможности избегаем
- **NoExecute** — существующие Pod-ы тоже будут evicted

Как это читать

Node с taint

node-1

gpu=true:NoSchedule

Нода говорит: **“без допуска не пущу”**

Pod без toleration → rejected

Pod с toleration

demo-pod

tolerates gpu=true

Pod говорит: **“этот запрет для меня допустим”**

Scheduler can place Pod here

NoSchedule

запретить новые Pod-ы

PreferNoSchedule

мягко избегать ноду

NoExecute

не только не ставить, но и выселять

Topology Spread Constraints

```
spec:
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule # или ScheduleAnyway
    labelSelector:
      matchLabels:
        app: my-app
```

Распределить Pod-ы равномерно по zones / nodes / racks.

- `maxSkew` — макс. разница между bucket-ами
- `topologyKey` — по чему делим (zone, hostname, rack)
- `whenUnsatisfiable: DoNotSchedule` — hard rule

Зачем нужен autoscaling

Проблема ручного sizing:

- пик нагрузки → сервису **не хватает capacity**
- тихие часы → кластер **жжёт деньги впустую**
- один workload упирается в **число Pod-ов**
- другой — в **размер CPU / memory** на Pod

Отсюда два основных подхода:

- **HPA** — масштабирует **в ширину**: больше / меньше Pod-ов
- **VPA** — масштабирует **в высоту**: больше / меньше ресурсов на Pod

На практике:

- stateless web/API чаще начинает с **HPA**
- batch / event-driven часто усиливается **KEDA**
- дефицит нод закрывает **Cluster Autoscaler**

Capacity vs demand

Without autoscaling

Fixed capacity

```
<svg viewBox="0 0 220 140" style="position:absolute;
inset:0; width:100%; height:100%; pointer-events:none;">
<path d="M8 112 C40 108, 58 78, 92 70 S152 18, 208 26"
fill="none" stroke="#dc2626" stroke-width="4"></path>
</svg>
```

peak = overload

time

На пике теряем трафик, внизу графика платим за простаивающий запас.

With autoscaling

Adaptive capacity

```
<svg viewBox="0 0 220 140" style="position:absolute;
inset:0; width:100%; height:100%; pointer-events:none;">
<path d="M8 112 C40 108, 58 78, 92 70 S152 18, 208 26"
fill="none" stroke="#dc2626" stroke-width="4"></path>
<path d="M8 116 L52 116 L52 96 L96 96 L96 74 L140 74
```


HPA — Horizontal Pod Autoscaler

- Меняет **число реплик** у Deployment / StatefulSet
- Работает по **CPU / memory / custom / external metrics**
- Требуется **metrics-server** или **Prometheus Adapter**
- Хорошо подходит для **stateless** web/API workload-ов
- Если нод не хватает, нужен ещё **Cluster Autoscaler**

```
spec:
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

HPA scales horizontally

Observe metrics

CPU target: 70%

current CPU: **92%**

HPA controller computes desired replicas

scale out when metric > target

Change replica count

Deployment: my-app

2 Pods → 5 Pods

same Pod size, more instances

HPA overview — как это работает

Metrics source

metrics-server

CPU / memory

or Prometheus Adapter for custom metrics

HPA controller

Compares actual vs target

current CPU = **92%**

target CPU = **70%**

desired replicas = **5**

updates Deployment scale

Workload

Deployment / ReplicaSet

2 Pods → 5 Pods

metrics

→

decision

→

more / fewer Pods

KEDA — event-driven autoscaling поверх HPA

- KEDA добавляет autoscaling по внешним событиям и external metrics
- Умеет масштабировать **Deployment**, **StatefulSet** и CR с `/scale`
- Главная суперсила: **scale to zero**
- Под капотом KEDA мониторит источник сигнала и передаёт метрики в Kubernetes / HPA

Event source

Prometheus / Queue / Kafka

→

KEDA

Operator + Metrics Server

→

HPA / Scale subresource

0 ↔ 1 ↔ N

Две фазы масштабирования в KEDA

- 1. **Activation phase** — KEDA решает, нужно ли выходить из 0 в 1
- 2. **Scaling phase** — дальше уже HPA решает масштабирование 1 ↔ N

Фаза	Кто принимает решение	Ключевой порог
0 ↔ 1	KEDA operator	activationThreshold
1 ↔ N	HPA controller	threshold / target

Если `minReplicaCount >= 1`, логика activation фактически теряет смысл: workload уже всегда активен.

KEDA + Prometheus scaler

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: prometheus-scaledobject
spec:
  scaleTargetRef:
    name: my-deployment
  minReplicaCount: 0
  maxReplicaCount: 12
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://prometheus:9090
      query: sum(rate(http_requests_total{deployment="my-deployment"}[2m]))
      threshold: "100"
      activationThreshold: "5"
```

- `query` должна вернуть **vector/scalar** с одним элементом
- `threshold` — цель для scale 1 ↔ N
- `activationThreshold` — порог для пробуждения 0 ↔ 1
- `serverAddress` — адрес Prometheus или Prometheus-compatible API

Аутентификация и production-нюансы для Prometheus scaler

Auth modes

- `bearer`
- `basic`
- `tls`
- `custom`

Через `TriggerAuthentication` можно собрать несколько режимов сразу, например `tls,basic`.

Важные опции

- `ignoreNullValues`
- `customHeaders`
- `queryParameters`
- `unsafeSsl`
- `timeout`

```
authenticationRef:  
  name: keda-prom-creds
```

VPA — Vertical Pod Autoscaler

- Меняет **requests/limits** Pod-ов на основе реального потребления
- 3 режима: `Off`, `Initial`, `Auto`
- В Kubernetes **1.33+** основной путь для CPU/memory — **in-place resize**
- Pod **не обязан** перезапускаться, если ноды и runtime поддерживают изменение
- ⚠ Не комбинируйте **VPA + HPA** по тем же метрикам (`CPU` / `memory`)

Современная модель: сначала попытка **resize без termination**, recreate — только fallback.

VPA scales vertically

Observe usage

Single running Pod

CPU request: **300m**

Memory request: **512Mi**

VPA sees sustained pressure

recommend bigger CPU / memory

Resize Pod resources

Same Pod, new resources

CPU request: **300m → 700m**

Memory request: **512Mi → 1Gi**

K8s 1.33+: apply in place when possible

preferred: no restart

fallback: recreate

VPA modes и внутренние компоненты

Режимы

- **Off** — только рекомендации
- **Initial** — выставляет ресурсы при создании Pod
- **Auto** — применяет рекомендации; в K8s 1.33+ предпочитает in-place resize, а recreate оставляет как fallback

Компоненты

- **Recommender** — считает target requests/limits
- **Updater** — решает, можно ли применить resize in place или нужен fallback
- **Admission Controller** — подставляет новые ресурсы при старте Pod

In-place Pod Resize (K8s 1.33+)

- Позволяет менять **CPU** и **memory** у уже работающего Pod
- kubelet проверяет, хватает ли ресурсов на нодe
- Если не хватает — Pod может получить статус **PodResizePending**
- Если изменение применяется — увидите **PodResizeInProgress**
- Требуется поддержки со стороны **CRI/runtime**

Это снижает disruption для VPA, но не отменяет архитектурные ограничения workload-a.

DaemonSet

Гарантирует, что **на каждой ноде** работает по одному Pod.

Типовые use cases:

- Log collector (Fluent Bit, Vector)
- Monitoring agent (Node Exporter, Datadog agent)
- CNI plugin (Cilium, Calico)
- Storage daemon (Rook OSD, OpenEBS)

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: logging
spec:
  selector:
    matchLabels: {app: fluent-bit}
  template:
    metadata:
      labels: {app: fluent-bit}
    spec:
      tolerations:
        - operator: Exists          # на BCEX нодах, включая tainted
      containers:
        - name: fluent-bit
          image: fluent/fluent-bit:2.0
```

DaemonSet overview — один Pod на каждую ноду

Node A

Fluent Bit Pod

logs / metrics / system hooks

Node B

Fluent Bit Pod

тот же агент, тот же template

Node C

Fluent Bit Pod

есть нода → есть Pod

When a node is added

DaemonSet controller creates one more Pod automatically

↔

When a node is removed

Pod on that node disappears with it; no replica balancing logic

Good for

agents, collectors, node-local services

Not for

stateless app replicas behind a Service

Job — одноразовая задача

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
spec:
  parallelism: 1
  completions: 1
  backoffLimit: 3
  activeDeadlineSeconds: 600
  ttlSecondsAfterFinished: 3600      # автоудаление через час
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: migrate
        image: my-app:migrate
        command: ["/migrate.sh"]
```

Use cases:

- Database migrations
- Batch обработка данных
- Backup задачи

Job: parallelism, retries и cleanup

- **parallelism** — сколько Pod-ов можно выполнять одновременно
- **completions** — сколько успешных завершений нужно всего
- **backoffLimit** — сколько раз пробовать заново после сбоя
- **activeDeadlineSeconds** — общий таймаут задачи
- **ttlSecondsAfterFinished** — автоочистка завершённых Job

```
spec:  
  parallelism: 5  
  completions: 50  
  backoffLimit: 4  
  ttlSecondsAfterFinished: 3600
```

CronJob — Job по расписанию

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"           # каждый день в 02:00
  concurrencyPolicy: Forbid       # не запускать если предыдущий ещё работает
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: backup-tool:1.0
```

`concurrencyPolicy` : `Allow` (по умолчанию), `Forbid` , `Replace` .

CronJob: основные параметры и реальные сценарии

- **schedule** — cron-выражение
- **jobTemplate** — какой Job создавать
- **concurrencyPolicy** — `Allow`, `Forbid`, `Replace`
- **successfulJobsHistoryLimit / failedJobsHistoryLimit** — сколько истории хранить
- **timeZone** — явно указывайте, если UTC вам не подходит

Типовые use cases:

- резервные копии;
- синхронизация данных;
- ротация / очистка;
- периодические отчёты и batch maintenance.

Time to make hands dirty — Lab 19

Taints, Tolerations, Topology Spread

Затаиним ноду, добавим toleration, сделаем zone-aware распределение.

 **labs/lab-19-scheduling.md**

Time to make hands dirty — Lab 20

HPA — Horizontal Pod Autoscaler в действии

Установим metrics-server, создадим HPA, нагрузим CPU, посмотрим autoscaling.

 **labs/lab-20-hpa.md**

Time to make hands dirty — Lab 21

Batch workloads — Job + CronJob

Запустим параллельный Job, потом расписание через CronJob.

 **labs/lab-21-job-cronjob.md**

Что дальше?

Production-grade K8s

- **Helm** — пакетный менеджер (Day 4)
- **GitOps** — Argo CD / Flux
- **Service mesh** — Istio / Linkerd / Cilium

Observability

- **Prometheus + Grafana** (Day 3)
- **EFK / Loki**
- **Jaeger / Tempo** — distributed tracing

CI/CD на K8s

- **GitLab CI/CD** (Day 5)
- **Tekton, Argo Workflows**
- **Image build: Kaniko, Buildah, BuildKit**

Спасибо!

Kubernetes в production — это про системное мышление

Kubernetes is the operating system of the cloud.

Master the API — and the cloud is yours.